PROJECT PRESENTATION & TECHNICAL DOSSIER

Equilibrium

AI-Powered Autonomous Student Workload Balancing Platform

Core Paradigm: Plan → Execute → Observe → Learn → Rebalance

Architecture: Microservices (React + Node.js + Python ML + MongoDB)

Target: >80% Sustainable On-Time Completion

1. Executive Summary & The Pitch

The 30-Second Elevator Pitch

"Conventional calendar apps fail students because they are completely static: when reality causes a 1-hour study session to run for 2.5 hours, the entire timetable collapses. Students suffer from severe optimism bias (underestimating durations by 30–50%), fall behind, sacrifice their sleep, and burn out. **Equilibrium is a closed-loop autonomous workload management platform** that continuously learns individual student estimation bias via machine learning, enforces circadian sleep and rest protections as unbreachable constraints, and dynamically recalculates the whole day in real-time the moment any delay occurs."

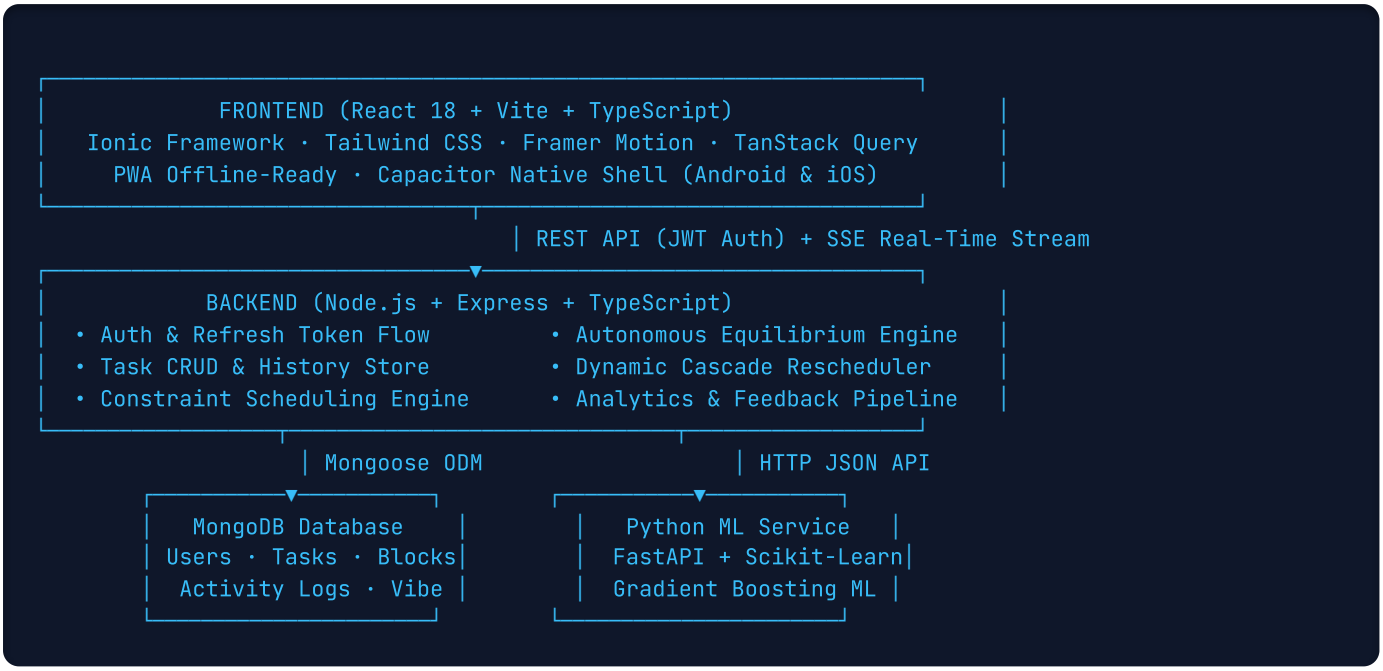
> 80%	3-Tier	15 Min	100%
NORTH STAR METRIC	INCREMENTAL ML	AUTONOMOUS CRON	SLEEP PROTECTED

2. Problem Statement: Why Existing Tools Fail

Traditional Apps (Google Cal, Todoist)	The Student Reality	Equilibrium Solution
Rigid, Static Slots: Fixed blocks with zero awareness of delays.	Delays cascade; students abandon the calendar by day 2.	**Dynamic Cascade Rescheduling:** Auto-shifts downstream blocks in real-time via Server-Sent Events (SSE).
Optimism Bias Ignored: Assumes user manual estimates are 100% accurate.	Students chronically underestimate programming, labs, and research tasks by 35–45%.	**3-Tier ML Predictor:** Corrects estimates automatically based on category, difficulty, and personal history.
Zero Sleep Guard: Allows scheduling study at 3:00 AM.	Sleep deprivation destroys retention and mental equilibrium.	**Circadian Lockout:** Hard-coded sleep and quiet hours that the algorithm strictly refuses to violate.
No Stress Feedback: Only tracks checkmarks.	No early warning before catastrophic academic burnout.	**Real-Time Workload Meter:** Visual index of load vs capacity + post-task vibe reflections.

3. System Architecture & Technical Stack

Equilibrium implements a distributed, decoupled 3-tier system built for high throughput and modular AI scaling:



Layer	Technologies	Key Architectural Responsibilities
Frontend	React 18, Vite, TypeScript, Ionic, Tailwind, Framer Motion	Responsive Cyber-Glassmorphic HUD, Workload Meter gauge, interactive schedule timeline, natural language task input, mobile PWA.
Backend API	Node.js, Express, TypeScript, Mongoose, Node-Cron	JWT authentication, business logic, heuristic scheduler, real-time SSE push server, 15-minute background equilibrium daemon.
ML Service	Python 3.11, FastAPI, Scikit-Learn, Joblib, NumPy	Task duration inference, cyclic temporal feature transformations, cold/warm/hot incremental regression model.
Database	MongoDB (Atlas or Local)	Stores polymorphic documents: Users (with circadian prefs), Tasks, ScheduleBlocks, TaskHistory, ActivityLogs.

4. Core Mathematical & Heuristic Algorithms

A. Multi-Factor Urgency Scoring Formula

Tasks in the queue are prioritized for greedy slot allocation using an asymptotic deadline-sensitive equation:

```
UrgencyScore = (Priority × 30) + (200 / HoursToDeadline) + (Difficulty × 5)
```

Rationale: Priority sets baseline ranking; as the deadline draws near, `(200 / HoursToDeadline)` approaches infinity, ensuring overdue/imminent tasks dynamically jump to the front of the schedule.

B. 3-Tier Progressive ML Duration Predictor

To overcome the classic cold-start hurdle without requiring weeks of initial data:

Stage	Trigger	Method & Logic
1. COLD START	< 10 completed tasks	**Rule-Based Multipliers:** Coding (1.35×), Projects (1.40×), Writing (1.25×), scaled by difficulty factor (0.90× to 1.20×).
2. WARM PATH	10 to 30 tasks	**Historical Blending:** Weighted average merging user's manual estimate with personal historical completion average.
3. HOT PATH	> 30 tasks	**Trained GradientBoostingRegressor:** Evaluates 12 features including cyclic trigonometric time-of-day/day-of-week encoding.

C. Heuristic Scheduling Constraints & Buffers

- **Circadian Sleep Lock:** Hard boundary during user's configured bedtime/wake hours.
- **15% Recovery Buffers:** Automatically placed after task blocks to absorb minor overruns without cascading.
- **Configurable Focus Breaks:** Automatically inserts rest blocks (Pomodoro: 25m/5m; Deep Work: 90m/15m).
- **Daily Workload Ceiling:** Prevents scheduling beyond user's daily study limit (e.g. 8h max), rolling excess to subsequent days.

5. The Autonomous Background Equilibrium Engine

> **Background Cron Daemon (`backend/src/scheduler/equilibrium.ts`)**
>
> Equilibrium is active even when the user isn't clicking buttons. An autonomous cron runs every 15 minutes to preserve schedule health.

Routine	Condition / Trigger	System Action
Deviation Detection	Active task has slipped > 30 minutes past its scheduled start/finish.	Triggers auto-reschedule; shifts downstream tasks forward and notifies frontend via SSE.
Overload Alert	Upcoming 48-hour assigned workload exceeds 130% of daily capacity.	Emits an alert warning to the user's dashboard and flags potential overflow tasks.
ML Retraining	10 new task completions logged since last model build.	Calls FastAPI `/train` endpoint to incrementally retrain the gradient boosting model.
Midnight Generation	Runs daily at 23:45.	Pre-computes the next day's balanced schedule so it's ready when the student wakes up.

6. Slide-by-Slide Presentation Guide (For Tomorrow)

Slide #	Slide Title	Key Speaking Points & Talking Script
Slide 1	**Title & Hook**	"Good morning. This is Equilibrium — an AI-powered autonomous workload balancing system. Our motto is *Plan → Execute → Observe → Learn → Rebalance*."
Slide 2	**The Student Dilemma**	"Every student knows the feeling: you plan to study 2 hours, it takes 4 hours, your sleep is ruined, and the entire calendar is abandoned. Why? Static tools assume humans are robots."
Slide 3	**The Equilibrium Solution**	"Equilibrium treats scheduling as a dynamic control system. If a task overruns, it recalculates automatically. Most importantly: sleep and rest are non-negotiable boundaries."
Slide 4	**System Architecture**	"We engineered a modern stack: React 18 + Ionic PWA frontend, Node.js + Express backend with Server-Sent Events, and a dedicated Python FastAPI microservice for machine learning."
Slide 5	**The Brain: Heuristics & Scoring**	"Our scheduling engine runs on an urgency formula balancing priority, deadline proximity, and task difficulty, inserting 15% recovery buffers and circadian sleep locks."
Slide 6	**Machine Learning Predictor**	"We solved the cold-start problem with a 3-tier model: rule-based category multipliers for beginners, historical blending for intermediate users, and Gradient Boosting regression for veterans."
Slide 7	**Live Demonstration**	*(Switch to browser demo)* "Watch how adding a task predicts actual duration, schedules it between college and sleep, and how clicking 'Extend' instantly cascades future tasks."
Slide 8	**The North Star Metric**	"Our primary success metric isn't just tasks completed — it's the *Sustainable On-Time Completion Rate* (tasks done on time without violating sleep or wellness limits). Target: >80%."
Slide 9	**Future Roadmap & Q&A**	"Future versions will include native mobile push notifications, Google Calendar sync, and our newly designed futuristic Focus Mode with desktop app blocking. Thank you! Any questions?"

7. Step-by-Step Live Demo Execution Script

> **Demo Tip: Rehearse this 2-minute flow once before presenting**
>
> Follow these exact steps in order for maximum visual impact during your presentation:

1. **Open the Dashboard (`http://localhost:5173`):**
   Show the *Workload Meter* dial. Point out the balance indicator (e.g. "Optimal / Healthy"). Point out the *Up Next* card and the metrics pills (Remaining, Done, Hours).
2. **Highlight Sleep & Circadian Enforcement:**
   Scroll down to the *Today's Schedule* timeline. Point out the 🌙 **Sleep** block. Say: *"Notice how sleep is a protected block. The engine guarantees 8 hours of sleep as an unbreachable constraint."*
3. **Create a Task with Natural Language:**
   Click "+ Add Task" or use the quick input bar. Enter: `"Machine Learning Lab Assignment"`, 3 hours, High Priority. Show that the AI predicted duration is automatically calculated.
4. **Navigate to Schedule Page:**
   Show how the system positioned the task between college lectures and bedtime, inserting a 🧉 break and 🔁 buffer.
5. **Trigger Dynamic Cascade Rescheduling:**
   Click "Extend (+30m)" on the active block. Point out how downstream tasks slide forward smoothly without manual calendar editing!

8. Viva / Q&A Defense: Tough Questions & Winning Answers

Q1: "Why did you separate the ML service into Python instead of doing everything in Node.js?"

Answer: "Separation of concerns and scalability. Node.js is optimal for high-concurrency I/O, routing, and real-time Server-Sent Events. Python is the industry benchmark for scientific computing and scikit-learn. By decoupling ML into a standalone FastAPI microservice, model retraining does not block the Node event loop, and both services can scale independently in Docker."

Q2: "What if a student has an impossible workload that physically cannot fit before the deadline?"

Answer: "Equilibrium strictly refuses to violate the student's sleep schedule to squeeze tasks in. Instead, the engine classifies these tasks as *Overflow Tasks*, sounds an overload warning on the Workload Meter, and advises the student to either negotiate extensions or reprioritize before the deadline is missed."

Q3: "How does the ML model avoid bad predictions when a user only has 1 or 2 tasks in the system?"

Answer: "We engineered a 3-tier progressive predictor specifically to solve cold start. Under 10 tasks, it relies on empirically proven category multipliers (e.g. coding tasks typically take 35% longer). From 10 to 30 tasks, it blends user estimates with personal averages. Only after 30 tasks does the gradient boosting regressor take full control."

Q4: "Can this system run on mobile devices?"

Answer: "Yes. The frontend is built using Ionic components with Capacitor configuration and full Progressive Web App (PWA) manifest support. It runs responsively in any mobile browser or can be packaged directly as native Android and iOS apps."